

transaction to gather all the information we need to make the call to the user's account. For example, if the user wants to make a deposit ❷, the program asks for the account number, the amount to deposit, and the password for the account ❸. We use the account number as a key into the dictionary of `Account` objects to get the appropriate `Account` object ❹. With that object, we then call the `deposit()` method, passing in the amount to deposit and the user's password ❺.

Once again, we have modified code at the main code level, and left our `Account` class untouched.

Creating an Object Manager Object

The code in [Listing 4-6](#) is actually doing two different things. The program first provides a simple menu interface. Then, when an action is chosen, it collects data and makes a call to a method of an `Account` object. Rather than having one large main program that does two different tasks, we can split this code into two smaller logical units, each with a clearly defined role. The menuing system becomes the main code that decides what action to take, and the rest of the code deals with the things a bank actually does. The bank can be modeled as an object that manages other (account) objects, known as an *object manager object*.

object manager object

An object that maintains a list or dictionary of managed objects (typically of a single class) and calls methods of those objects.

This split can be made easily and logically: we take all the code related to the bank and put it into a new `Bank` class. Then, at the beginning of the main program, we instantiate a single `Bank` object from the new `Bank` class.

The `Bank` class will manage a list or dictionary of `Account` objects. In this way, the `Bank` object will be the only code that communicates directly with `Account` objects ([Figure 4-1](#)).